

MÉTRICAS DE CALIDAD Y RENDIMIENTO: EVALUACIÓN DE PROGRAMACIÓN CONCURRENTE DESARROLLADA MEDIANTE VIBECODING

Integrantes:

Nahuel DE LA CRUZ ⁽¹⁾

adelacruzamudio@alumno.unlam.edu.ar

Tomás FELICE ⁽²⁾

tfelice@alumno.unlam.edu.ar

Brian GRANEROS ⁽³⁾

bgraneros@alumno.unlam.edu.ar

Asignatura:

(3658) PROGRAMACIÓN CONCURRENTE

Fecha - Versión:

27/05/2026 – Versión 1.01

Resumen:

El presente artículo académico expone una evaluación integral de calidad y rendimiento sobre un software concurrente desarrollado de forma automatizada mediante herramientas de VibeCoding. Utilizando la plataforma Claude Code con el modelo masivo Claude Opus 4.7, se generó un pipeline multihilo en Python enfocado en el procesamiento masivo de imágenes bajo el patrón de diseño productor-consumidor con colas acotadas. Posteriormente, la estructura y el comportamiento dinámico del código fueron auditados mediante la aplicación de métricas estáticas y de entorno, empleando herramientas como Radon y utilidades del sistema operativo Linux. Los hallazgos cuantitativos revelaron niveles óptimos de mantenibilidad y un paralelismo real efectivo en el uso de la CPU. No obstante, el análisis crítico evidenció limitaciones en la robustez del sistema fuera de los escenarios ideales de ejecución.

Palabras Clave: Programación concurrente, VibeCoding, Métricas de software, Calidad de código, Paralelismo.

I. CONTEXTO

El presente trabajo de investigación se desarrolla dentro del área del conocimiento de la Ingeniería de Software y las Ciencias de la Computación, abordando específicamente la temática de la Programación Concurrente y Paralela orientada al procesamiento distribuido de datos. Este proyecto se encuentra coordinado institucionalmente por el Departamento de Ingeniería e Investigaciones Tecnológicas (DIIT) de la Universidad Nacional de La Matanza (UNLaM), en el marco de las actividades prácticas obligatorias dispuestas para la formación de grado de los estudiantes de la carrera de Ingeniería en Informática durante el primer cuatrimestre del año 2026. La investigación posee un carácter netamente académico y experimental, enfocada en auditar de forma empírica la viabilidad, mantenibilidad y eficiencia del paradigma de generación automatizada de software (VibeCoding) frente a la resolución de problemas concurrentes de complejidad no trivial que exigen mecanismos estrictos de sincronización y control de recursos.

II. INTRODUCCIÓN

El objetivo del presente trabajo consiste en experimentar con la generación automatizada de código mediante herramientas de *VibeCoding* para resolver un problema concurrente de complejidad no trivial, ejecutando posteriormente un análisis crítico de su calidad y rendimiento a través de la aplicación de métricas de software de ingeniería. Específicamente, se evalúa la capacidad de un modelo de lenguaje masivo (LLM) para estructurar un pipeline multihilo eficiente en Python bajo pautas de diseño y restricciones arquitectónicas estrictas.

III. DESARROLLO

A. Prompt Elaborado

LENGUAJE DE PROGRAMACIÓN

- Utilizar Python 3.11+ con anotaciones de tipo, siguiendo el Google Python Style Guide adaptado a la cátedra.

PLATAFORMA

- Desarrollar una aplicación CPU-only multihilo ejecutable en Linux/macOS/Windows, distribuida como paquete Python (``parallelvision``), invocable por línea de comandos vía ``python -m parallelvision.main``.
- Dependencias permitidas (únicas, declaradas en ``requirements.txt``): ``pillow``, ``opencv-python``, ``numpy``, ``pylint``, ``pytest``. Toda la concurrencia se resuelve con la biblioteca estándar (``threading``, ``queue``, ``concurrent.futures``).

FUNCIONALIDAD A IMPLEMENTAR

- Desarrollar un pipeline concurrente de procesamiento masivo de imágenes que recorra una carpeta de entrada, distribuya el trabajo entre un pool de workers, aplique una operación de transformación a cada imagen y consolide métricas de rendimiento por operación de manera thread-safe, siguiendo el patrón productor-consumidor con cola acotada y agregador dedicado.

REQUISITOS ESPECÍFICOS

- Se debe implementar la siguiente estructura de paquete, sin agregar archivos extra:

```

'''text
parallelvision/
├─ core/
│   ├── __init__.py
│   ├── backend.py          # CPUBackend + get_backend() (solo CPU)
│   ├── queue_manager.py    # ImageQueue
│   └─ metrics.py          # MetricsCollector
├─ pipeline/
│   ├── __init__.py
│   ├── image_loader.py     # Productor: escanea carpeta, encola paths
│   ├── worker.py           # ProcessingWorker + ThreadPoolExecutor
│   └─ result_aggregator.py # Consumidor de resultados → MetricsCollector
└─ main.py                  # Arma el pipeline, lo corre, imprime resumen
'''

```

- Se debe implementar una clase `ImageQueue` en `core/queue_manager.py` con constructor `ImageQueue(maxsize: int)` y los métodos públicos `put(item)`, `get()`, `task_done()`, `join()`, `empty()` y `qsize()`. Internamente debe envolver `queue.Queue` sin exponer la cola interna.

- Se debe implementar una clase `CPUBackend` en `core/backend.py` con un único método público `process(image: np.ndarray, operation: str) -> np.ndarray` que soporte exactamente cuatro operaciones:

Operación	Implementación esperada
-----	-----
<code>`grayscale`</code>	Conversión a escala de grises (OpenCV o Pillow).
<code>`edges`</code>	Detección de bordes con Canny (OpenCV).
<code>`blur`</code>	Gaussian Blur con kernel configurable (OpenCV).
<code>`equalize`</code>	Ecualización de histograma (OpenCV).

- Se debe implementar la función `get_backend() -> CPUBackend` en `core/backend.py` que por ahora retorna siempre `CPUBackend()`, dejando un comentario `# TODO: detección CUDA/OpenCL en futura iteración`.

- Se debe implementar la clase `MetricsCollector` en `core/metrics.py` con los métodos `record(image_name: str, operation: str, elapsed_ms: float)` (protegido con `threading.Lock`), `get_summary() -> dict` (devolviendo por operación: ``count``, ``avg_ms``, ``min_ms``, ``max_ms``) y `get_throughput(total_seconds: float) -> float` (imágenes/segundo global).

- Se debe implementar el módulo ``pipeline/image_loader.py`` con la función ``scan_folder(input_dir: str) -> list[str]`` que filtre por extensiones válidas (constante ``IMAGE_EXTENSIONS``), y la función ``enqueue_paths(paths: list[str], queue: ImageQueue, num_workers: int)`` que encole los paths y publique un sentinela ``None`` por cada worker.
- Se debe implementar la clase ``ProcessingWorker`` en ``pipeline/worker.py`` que reciba en su constructor ``input_queue``, ``result_queue``, ``backend`` y ``operation``. Su método ``run()`` debe consumir hasta sentinela ``None``, medir el tiempo con ``time.perf_counter()`` antes y después de ``backend.process()``, y encolar la tupla ``(image_name, operation, elapsed_ms)`` en la cola de resultados. Los workers deben lanzarse vía ``concurrent.futures.ThreadPoolExecutor(max_workers=MAX_WORKER_THREADS)``.
- Se debe implementar el módulo ``pipeline/result_aggregator.py`` como un hilo dedicado que consuma la cola de resultados hasta recibir un sentinela ``None`` y llame a ``MetricsCollector.record()`` por cada resultado.
- Se debe implementar ``main.py`` que parsee los argumentos ``--input-dir``, ``--operation``, ``--workers`` (default 4) y ``--queue-size`` (default 32), arme el pipeline, lo arranque, espere su finalización e imprima el resumen formateado.
- Se debe garantizar shutdown limpio: workers y agregador deben drenar las colas y finalizar sin colgarse, usando sentinelas ``None`` o ``queue.join()`` (a criterio de quien implementa, justificando la elección con un comentario breve). El programa debe llamar a ``executor.shutdown(wait=True)``.
- Se debe usar ``concurrent.futures.ThreadPoolExecutor`` con número de workers configurable mediante constante ``MAX_WORKER_THREADS``.
- Se debe usar ``queue.Queue`` thread-safe acotada (``maxsize`` configurable mediante constante ``DEFAULT_QUEUE_SIZE``) para la cola de entrada y la cola de resultados.
- Se debe usar ``threading.Lock`` para proteger el estado interno de ``MetricsCollector``.
- El sistema debe imprimir, al finalizar, un resumen con el siguiente formato:

```

```text
[INFO] Backend activo: CPUBackend
[INFO] Procesadas 50 imágenes en 3.21 s (15.58 img/s)
Resumen por operación:
 grayscale → count=50 avg=64.2 ms min=41.0 ms max=128.5 ms
```

```

- Se debe usar ``logging.info('msg: %s', var)`` para todas las trazas. Nunca f-strings dentro de logging.
- Se debe levantar ``ValueError`` en ``CPUBackend.process()`` si la operación recibida no pertenece al conjunto ``VALID_OPERATIONS``.

RESTRICCIONES

- Máximo 15 líneas por función o método (sin contar firma ni docstring). Esta regla es de la cátedra y no admite excepciones.
- Sin números mágicos. Toda constante numérica o string fijo debe declararse como ``CAPS_WITH_UNDER`` al tope del módulo. Constantes obligatorias mínimas: ``IMAGE_EXTENSIONS``, ``VALID_OPERATIONS``, ``MAX_WORKER_THREADS``, ``DEFAULT_QUEUE_SIZE``, ``GAUSSIAN_KERNEL_SIZE``, ``CANNY_LOW_THRESHOLD``, ``CANNY_HIGH_THRESHOLD``.
- Línea máxima: 80 caracteres. Sin backslash de continuación; usar paréntesis. Indentación: 4 espacios. Sin punto y coma al final de línea.
- Imports absolutos (``from parallelvision.core import ...``), nunca relativos. Orden: ``__future__`` → `stdlib` → `terceros` → `proyecto`, separados por línea en blanco.
- Type hints obligatorios en toda la API pública. Docstrings estilo Google (``Args:`` / ``Returns:`` / ``Raises:``) en toda función pública o de más de 10 líneas.
- Mutable defaults prohibidos. Usar ``None`` y asignar dentro de la función.
- Excepciones específicas. Nunca ``except:`` ni ``except Exception: pass``.
- Nomenclatura obligatoria: clases en ``CapWords``; funciones, métodos y variables en ``lower_with_under``; constantes en ``CAPS_WITH_UNDER``; identificadores privados con prefijo ``_``.
- El código debe pasar ``pylint`` con ``max-line-length=80`` y ``max-module-lines=200`` sin warnings no justificados.
- Concurrencia obligatoria con la biblioteca estándar. Está prohibido usar ``multiprocessing``, ``numba``, ``pyopencl``, frameworks de GUI, ni librerías de paralelismo de alto nivel (``joblib``, ``dask``, ``ray``, ``asyncio``, etc.).
- Toda comunicación entre hilos debe pasar por ``queue.Queue``. Todo acceso a estado compartido debe pasar por ``threading.Lock``. No asumir atomicidad de built-ins.
- Sin race conditions: ninguna operación read-modify-write sobre estado compartido fuera del ``Lock``.
- Sin deadlocks: orden de adquisición de recursos consistente; los sentinelas deben garantizar que todos los hilos finalicen.
- Sin busy-wait: está prohibido cualquier bucle de polling con ``sleep``. Usar ``Queue.get()`` bloqueante o ``Event.wait()``.
- Exclusión mutua estricta: sólo un hilo puede modificar el dict de métricas a la vez.

- NO se debe generar: backends GPU (CUDA, OpenCL, Numba), detección dinámica de hardware, GUI / dashboard / matplotlib, exportación a CSV, tests unitarios, ni manejo de configuración por archivo (YAML, TOML, JSON).
- La salida de la generación debe ser un único bloque de código por archivo, en el orden de la estructura de paquete declarada arriba, precedido por el path relativo del archivo. Sin explicaciones intercaladas. Al final, un único párrafo de máximo 5 líneas describiendo cómo se garantiza la ausencia de deadlocks y por qué se eligió el mecanismo de finalización (sentinela vs `join`).

B. Herramienta de VibeCoding utilizada

Para la generación automatizada del software concurrente se seleccionó la interfaz de línea de comandos Claude Code, una plataforma orientada a la edición directa de proyectos mediante agentes de inteligencia artificial. Esta herramienta fue integrada al entorno de desarrollo local para interactuar de forma nativa con la estructura del repositorio y automatizar la creación de los módulos concurrentes.

C. Modelo de IA utilizado

Como motor principal de procesamiento de lenguaje natural y generación de código se utilizó el modelo masivo Claude Opus 4.7. La elección de este modelo se fundamentó en su capacidad avanzada para interpretar restricciones técnicas complejas, resolver lógicas de sincronización de hilos y cumplir con los estándares de diseño y estilo de codificación exigidos.

D. Código generado

Con el propósito de garantizar la reproducibilidad del experimento, el código fuente base del pipeline concurrente quedó registrado y versionado de forma oficial en el repositorio asignado para el proyecto, cuyo acceso público se encuentra detallado en [\[9\]](#).

E. Análisis crítico del Código generado

Después de generar el código y realizar distintas pruebas con imágenes, se concluye la siguiente evaluación:

El código generado cumple con el objetivo planteado y con los requisitos pedidos en el prompt system.

El código genera el output esperado. También dió un instructivo de cómo realizar la ejecución para que los testers puedan realizar pruebas sin dificultad.

Se respetó la estructura de los archivos y las restricciones indicadas en el prompt y en la consigna del TP.

Analizando la implementación de mecanismos de concurrencia, se sacaron las siguientes conclusiones:

Se usa un ThreadPoolExecutor, para lanzar los workers en un pipeline y que, por cada una, encole las imagenes a procesar. Cada una lo va a realizar mediante este hilo. Se define la cantidad maxima de workers en el main.

Se crea la clase “ResultAggregator”, un hilo dedicado a consolidar los resultados de las metricas que saque por cada imagen procesada.

La comunicación de los hilos se hace por medio de un mecanismo patron-consumidor usando las colas como la herramienta principal y sus funciones de comunicacion “queue.join()” y “task_done”, desde la clase “ImageQueue” que se encuentra en el archivo “core\queue_manager.py”. También usa “threading.Lock()” en el caso del ThreadPoolExecutor.

Errores y limitaciones encontradas:

Introduce fallos de seguridad que se contemplan despues de leer el codigo generado. Por ejemplo: la cantidad de workers realmente creados debe ser igual al argumento “args.workers”. Si un worker muere por una excepcion no controlada antes de leer su sentinela, quedaria un worker = None sin consumir. Es decir que la igualdad de conteos es implicita, y no verificada por el codigo, lo cual lo hace frágil ante cambios o baja mantenibilidad si el codigo sigue creciendo.

Es dificil depurar el código y encontrar algún error fuera de los términos que se le pidió por el system prompt. Ya que el código fue diseñado para mostrarte y ejecutar los caminos felices, pero no contempla escenarios adversos (por ejemplo, la existencia del directorio de las imagenes que se van a procesar).

IV. RESULTADOS

Para evaluar los resultados del código generado por el modelo de IA utilizado, se utilizaron diferentes métricas.

A. Métricas utilizadas

1. Complejidad ciclomática

Fundamento teórico: La Complejidad Ciclomática debe utilizarse como métrica de medición del código porque permite cuantificar el nivel de complejidad lógica de un programa a partir de la cantidad de caminos de ejecución posibles. Según la documentación oficial de Microsoft Learn, esta métrica se basa en contar las decisiones presentes en el código, como estructuras if, switch, for o while. A mayor cantidad de decisiones, mayor será la complejidad del software.

El principal fundamento teórico para utilizar esta métrica es que existe una relación directa entre la complejidad del código y la dificultad para:

- comprenderlo,
- mantenerlo,
- probarlo,
- y detectar errores.

Mecanismo de medición: Radon CC.

Radon es una herramienta de Python para generar informes sobre métricas de código, incluyendo la complejidad ciclomática y el índice de mantenibilidad. [\[1\]](#) [\[2\]](#) [\[3\]](#)

Entre sus ventajas, se encuentran:

- Es reproducible y objetivo. No requiere criterio humano.
- Implementa el algoritmo de McCabe estándar sobre el AST (Árbol de Sintaxis Abstracta) de Python, sin depender del estilo del código.
- Da el grado por bloque (función/método/clase), que es la granularidad correcta — la CC de un módulo entero no es accionable. A continuación, en la Fig. 1, se muestra la tabla con los rangos, y la puntuación para cada uno.

| Puntuación
CC | Rango | Riesgo |
|------------------|-------|--|
| 1 - 5 | A | Bloque bajo - simple |
| 6 - 10 | B | Bloque bajo - bien estructurado y estable |
| 11 - 20 | C | Bloqueo moderado - ligeramente complejo |
| 21 - 30 | D | Más que moderado - bloque más complejo |
| 31 - 40 | E | Bloque alto - complejo, alarmante |
| 41+ | F | Bloque inestable muy alto - propenso a errores |

Fig 1. Escala de valoración de la Complejidad Ciclomática (CC) y su asociación con los niveles de riesgo y estabilidad del bloque de código.

Resultados obtenidos: En la Fig. 2 se observan los resultados obtenidos.

| Bloque | CC | Rango |
|---------------------------------|-----|-------|
| --- | --- | --- |
| `CPUBackend` (clase) | 3 | A |
| `ResultAggregator.run` | 3 | A |
| `ProcessingWorker.run` | 3 | A |
| `scan_folder` / `enqueue_paths` | 3 | A |
| `main._print_summary` | 3 | A |
| Resto (28 bloques) | 1-2 | A |

Fig. 2. Distribución de los bloques principales del sistema y su clasificación en el rango de riesgo A según su Complejidad Ciclomática.

Máximo absoluto: CC = 3. No hay un solo bloque por encima del rango A.

2. Mantenibilidad

Fundamento teórico: El Índice de mantenibilidad o Maintainability Index (MI) es un índice compuesto que combina tres señales [4]:

$$MI = 171 - 5.2 \cdot \ln(V) - 0.23 \cdot CC - 16.2 \cdot \ln(SLOC) [+ \text{factor de comentarios}]$$

donde V es el volumen de Halstead, CC la complejidad ciclomática y SLOC las líneas de código. Radon lo normaliza a una escala 0–100 (A: $\geq 20/100$ en su escala, presentada como porcentaje).

Es relevante porque intenta responder, en un solo número, "¿cuánto costará modificar este módulo en el futuro?". El costo de mantenimiento domina el costo total del software a lo largo de su vida.

En la Fig. 3, se pueden observar los umbrales que se manejan.

| Valor de índice | Color | Significado |
|-----------------|----------|---|
| 0-9 | Rojo | Baja capacidad de mantenimiento del código |
| 10-19 | Amarillo | Mantenimiento moderado del código |
| 20-100 | Verde | Buena capacidad de mantenimiento del código |

Fig. 3. Rangos de clasificación del Índice de Mantenibilidad (MI) y su semántica de color asociada a la capacidad de mantenimiento.

Mecanismo de medición: radon mi –s

- Radon es la implementación de facto del índice de mantenimiento en el ecosistema Python.
- Combina volumen, CC y tamaño en una sola lectura comparable entre módulos.

Radon utiliza otra derivada diferente a la original [5], observable en la Fig. 4.

$$MI = \max \left[0, 100 \frac{171 - 5.2 \ln V - 0.23G - 16.2 \ln L + 50 \sin(\sqrt{2.4C})}{171} \right]$$

Fig. 4. Ecuación estándar para el cálculo del Índice de Mantenibilidad (\$MIS) considerando volumen, complejidad y líneas de código.

Dónde:

- V Es el volumen de Halstead (ver más abajo);
- G Es la complejidad ciclomática total;
- L es el número de Líneas de Código Fuente (SLOC);
- C Es el porcentaje de líneas de comentarios (importante: convertido a radianes).

Resultados obtenidos: Todos los módulos en rango A. Mínimo: worker.py con 74.40.

Conclusión: El código generado tiene buena capacidad de mantenimiento del código.

3. Rendimiento – Cantidad de procesos y threads utilizados

El modelo de ejecución determina el techo de rendimiento alcanzable:

- **Procesos:** Espacios de memoria aislados; el SO los planifica en cualquier núcleo. En CPython evitan el GIL pero pagan costo de IPC y serialización.
- **Threads:** Comparten memoria; baratos de crear y comunicar. En CPython comparten el GIL (Global Interpreter Lock): solo un hilo ejecuta bytecode Python a la vez. El paralelismo real con hilos solo ocurre cuando el trabajo libera el GIL (I/O, o llamadas a librerías nativas que lo sueltan).

Medir cuántos procesos/threads usa el programa, y contra qué techo teórico, es lo que distingue "concurrente" (varias tareas en progreso) de "paralelo" (varias tareas ejecutando a la vez). [6]

Mecanismo de medición:

- Inspección estática del código — cuenta los hilos por construcción: ThreadPoolExecutor(max_workers=N) + el hilo ResultAggregator + el hilo principal. Justificación: el número de hilos del diseño es determinista, no hace falta instrumentar para conocerlo.
- /usr/bin/time -v — reporta "Percent of CPU this job got", que sobre 1 núcleo es 100 %. Un valor >100 % es prueba empírica de paralelismo real.
- Conteo de procesos: el programa nunca importa multiprocessing ni hace fork → un único proceso (verificable estáticamente).

Resultados obtenidos:

Para realizar la prueba, se utilizó la topología de ejecución de 1 proceso

| Hilo | Origen | Rol |
|-------------------|-----------------------------------|---|
| MainThread | intérprete | arma pipeline, produce paths, imprime resumen |
| 4 × worker | ThreadPoolExecutor(max_workers=4) | consumidores: leen y procesan imágenes |
| result-aggregator | threading.Thread | consumidor único de resultados → métricas |

Se realizó una prueba usando 4 imágenes.

Percent of CPU this job got: 304%

Elapsed (wall clock): 3.53 s • Pipeline interno: 1.85 s

4. Uso de CPU y memoria

- **CPU:** Separar user time (cómputo del programa) de system time (syscalls, I/O) y del wall clock. La relación (user+sys)/wall es el grado de paralelismo efectivo.

- **Memoria:** El RSS pico (Resident Set Size) es la memoria física máxima que el SO le asignó. Importa porque acota la escalabilidad: procesar imágenes carga arrays NumPy ($H \cdot W \cdot C$) bytes; con colas acotadas, el pico de memoria es predecible y ese es justamente un objetivo de diseño.

Mecanismo de medicion: `/usr/bin/time -v`

Time -v es un comando de linux que permite saber el uso máximo de memoria de un proceso. En este caso, es el "Tamaño máximo del conjunto residente", Aparte del uso de memoria, GNU time con la opción %P proporciona estadísticas no relacionadas (%CPU), que dependen del planificador y, por lo tanto, son bastante variables. [6]

Resultados obtenidos:

```
User time:      5.83 s
System time:    4.91 s
Wall clock:     3.53 s
% CPU:         304 %
Maximum RSS:    405 192 kB  ≈ 396 MB
Minor page faults: 235 863    ·    Major (I/O): 528
Context switches: 750 voluntarios / 2 290 involuntarios
```

5. Cantidad de tokens utilizados

En generación de código asistida por IA, el token es la unidad de cómputo y de costo. La cuenta de tokens importa porque [7]:

- Es proporcional al costo monetario y energético de generar el código.
- Mide la eficiencia del prompt: un prompt que produce código correcto con pocos tokens es más eficiente que uno que necesita muchas iteraciones largas.

Resultados obtenidos:

~2.216 tokens (116 de entrada y 2.100 de salida)

6. Cantidad de refinamientos

La cantidad de refinamientos de un prompt es una tarea involucrada con la ingeniería de prompts.

En el contexto de prompts, la cantidad de refinamientos representa la cantidad de iteraciones que se hicieron de modificar a un prompt con el objetivo de mejorar el resultado, es decir, el código generado.

Cada iteración implica técnicas para entrenar aún más a un modelo cuyos pesos ya se han actualizado mediante entrenamiento previo. También aprovecha los amplios conocimientos y la estabilidad adquiridos con las iteraciones anteriores. [8]

Mecanismo de medición seleccionado: historial de Git

El repositorio es la fuente de verdad versionada y, a falta de un log de sesión, los mensajes de commit son la única evidencia objetiva del ciclo de iteración.

Resultados obtenidos:

```
526b619 docs: Crear carpeta y agregar consigna de TP-Metricas-AI
f373d3d docs: Crear prompt para TP-Metricas-AI
639932b feat: Iteracion 1 TP-Metricas-AI      ← objeto de análisis
```

El commit se titula explícitamente "Iteracion 1".

No existen commits Iteracion 2, fix, ni refactor posteriores sobre estos archivos.

Refinamientos sobre el código entregado: 0 (una única iteración aceptada tal cual se generó).

7. Grado de correctitud del código en forma concurrente

Se evalúan cuatro propiedades: sincronización adecuada, ausencia de deadlocks, ausencia de race conditions y exclusión mutua.

Un programa concurrente es correcto si cumple propiedades de safety ("nada malo ocurre": sin race conditions, sin corrupción de estado) y de liveness ("algo bueno termina ocurriendo": sin deadlock, sin starvation, terminación). Estas propiedades no se garantizan testeando: un test pasa para un interleaving y hay exponencialmente muchos. Se garantizan por razonamiento sobre el diseño: invariantes, protocolos y disciplina de locks.

Mecanismo de medición:

Se usó revisión analítica del diseño (no una herramienta automática). Justificación:

- No existe para Python un verificador de modelos práctico que pruebe ausencia de deadlock/race en código real con OpenCV. Herramientas como ThreadSanitizer no cubren el modelo de hilos de CPython + GIL.
- La corrección concurrente se demuestra deductivamente: identificar el estado compartido, los puntos de sincronización y verificar invariantes.
- Como complemento empírico se usó la corrida de la Sección 3 (terminó sin colgarse ⇒ evidencia a favor de liveness, pero no prueba — un deadlock puede manifestarse 1 de cada 1000 corridas).

Resultados:

Sincronización adecuada

- Toda comunicación entre hilos pasa por ImageQueue (wrapper de queue.Queue), que es thread-safe por construcción. Hay dos colas: input_queue (productor → workers) y result_queue (workers → aggregator).
- No hay sleep de polling en ningún lado. Los hilos se bloquean en queue.get(), que es la primitiva correcta: sin busy-wait.

- El cierre del pool usa `executor.shutdown(wait=True)` → main no avanza hasta que los 4 workers terminaron. La sincronización entre fases es explícita y correcta.

Ausencia de deadlocks

Razonamiento:

- Un solo tipo de recurso bloqueante por hilo a la vez. Los workers bloquean en `input_queue.get()` o en `result_queue.put()`; el aggregator en `result_queue.get()`. Ningún hilo retiene un lock mientras espera otro ⇒ no hay espera circular (se rompe una condición de Coffman).
- Las colas acotadas podrían bloquear un put, pero el `result_queue` tiene un consumidor (`ResultAggregator`) que corre en paralelo y la drena; nunca queda llena permanentemente.
- Terminación garantizada por sentinelas: `enqueue_paths` publica exactamente un `None` por worker → cada worker consume su sentinela y sale. Tras `shutdown(wait=True)`, main publica un `None` en `result_queue` para el único aggregator. La cuenta de sentinelas coincide con la cuenta de consumidores.
- El `try/finally` en `run()` asegura `task_done()` aún si `process()` lanza excepción → no se pierde el conteo de la cola; un `join()` futuro no se colgaría.

Riesgo residual identificado (crítica): la corrección de la terminación depende de que `args.workers` (lo que se pasa a `ThreadPoolExecutor` y a `enqueue_paths`) sea el mismo número que la cantidad de workers realmente creados. Hoy lo es porque ambos usan `args.workers`. Pero si un worker muere por una excepción no capturada antes de leer su sentinela, quedaría un `None` sin consumir en `input_queue` — inofensivo —; el caso peligroso inverso (más consumidores que sentinelas) no ocurre con el código actual. El diseño es correcto, pero frágil ante cambios: la igualdad de conteos es un invariante implícito, no verificado por código.

Ausencia de race conditions

- El estado compartido mutable es uno solo: los diccionarios internos de `MetricsCollector` (`_samples`, `_total_count`).
- `record()` hace un read-modify-write (`setdefault + append, += 1`) íntegramente dentro de `with self._lock`. La consigna exige explícitamente que ninguna operación RMW sobre estado compartido quede fuera del lock — se cumple.
- `get_summary()` y `get_throughput()` también leen bajo el lock ⇒ no leen un dict a medio actualizar.
- El código no asume atomicidad de built-ins: `_total_count += 1` está protegido, aunque en CPython "parezca" atómico. Cumple la regla de la cátedra.
- Las imágenes NumPy se pasan por la cola y cada worker trabaja sobre su array; no hay un buffer compartido escrito por varios hilos.

Exclusión mutua

- `threading.Lock` garantiza que a lo sumo un hilo modifique el dict de métricas a la vez.
- Crítica importante: en el diseño actual la exclusión mutua está sub-ejercitada. `record()` es invocado solo por el `ResultAggregator`, que es un único hilo. Los workers no tocan `MetricsCollector`: depositan tuplas en `result_queue`. Por lo tanto, hoy no existe contención real sobre el lock — nunca hay dos hilos compitiendo por él en `record()`.
- Esto no es un defecto: es un patrón sólido (funnel a un consumidor único; serializa las escrituras por diseño, no por lock).
- Pero significa que el `threading.Lock` es defensivo / contractual, no funcionalmente necesario en el estado actual. Cumple la consigna al pie de la letra y protege ante una futura iteración donde varios hilos llamen a `record()`. Un análisis 100 % crítico debe decirlo: el TP "demuestra" exclusión mutua sobre un escenario que no la pone a prueba. La corrección es real, pero la evidencia empírica de exclusión mutua es nula porque el diseño elimina la concurrencia justo en ese punto.

B. Discusión de Hallazgos en las Métricas

A partir de los resultados cuantitativos obtenidos mediante las mediciones estáticas y dinámicas, se desprenden los siguientes hallazgos sobre la calidad y el comportamiento del código generado por la inteligencia artificial:

1. Eficiencia en el diseño y mantenibilidad

El hallazgo más contundente a nivel de código es que la totalidad de los módulos se ubicaron en el rango máximo de mantenibilidad (Rango A). Esto se correlaciona directamente con los resultados de la complejidad ciclomática, donde ningún bloque del sistema superó un valor de $CC = 3$. Al imponer una restricción estricta de un máximo de 15 líneas por función, la IA se vio obligada a modularizar el pipeline de manera extrema. Esto demuestra que un prompt con restricciones estructurales fuertes produce código con un índice de riesgo bajo, muy estable y con un costo de modificación a futuro prácticamente nulo.

2. Paralelismo real efectivo en Python

Los datos arrojados por la herramienta de medición dinámica indican un consumo de CPU del 304% sobre un entorno de ejecución multihilo. En el ecosistema de CPython, la existencia del GIL (Global Interpreter Lock) suele limitar la ejecución concurrente a un solo núcleo. Sin embargo, el hallazgo de un porcentaje netamente superior al 100% demuestra que el pipeline logró un paralelismo real.

3. Predictibilidad y control del consumo de memoria

El tamaño máximo del conjunto residente (Maximum RSS) se estabilizó en 396 MB procesando un lote de imágenes. El diseño del prompt exigió el uso de una estructura de productor-consumidor con una cola acotada. El sistema nunca

sufrirá un desbordamiento por cargar infinitas imágenes en memoria en paralelo, ya que el hilo productor se bloquea automáticamente si los hilos trabajadores no drenan la cola a tiempo.

4. Costo-eficiencia del proceso de VibeCoding

El desarrollo del software completo requirió el consumo aproximado de únicamente 2.216 tokens y se resolvió con 0 refinamientos posteriores (una sola iteración de prompt). Esto expone que el modelo de IA (Claude Opus 4.7) posee una alta efectividad para interpretar requisitos técnicos complejos y traducirlos a estructuras concurrentes válidas bajo estándares estrictos en un solo intento, lo que reduce drásticamente el costo monetario y temporal comparado con un desarrollo tradicional. Sin embargo, la ausencia de refinamientos dejó expuestas fallas críticas de robustez fuera del camino principal, como la falta de validación de rutas o la pérdida silenciosa de hilos ante excepciones.

V. CONCLUSIONES

El código generado cumple todos los requisitos funcionales del prompt, implementa con propiedad el patrón productor-consumidor, usa los mecanismos de concurrencia adecuados (colas thread-safe, Lock, sentinelas) y es concurrentemente correcto en el camino feliz — sin race conditions, sin deadlock y con exclusión mutua sobre el estado compartido.

Las ventajas que pudimos encontrar son las siguientes:

- Velocidad de generación de código con respecto a lo que haría un dev.
- Cumple con los requisitos funcionales que se le pide en el prompt
- Código funcional y con instructivo para poder realizar pruebas.
- Trazabilidad de los archivos creados.
- Explicación detallada de la utilidad de cada función y clase creada, y los criterios utilizados.

Sin embargo, se encontraron algunas limitaciones y fallas:

- Cuelgue ante error: el agregador no-daemon, sin sentinela garantizado por try/finally, viola el requisito explícito de "finalizar sin colgarse" (Verificado con --workers 0).
- Garantías de concurrencia evadibles: -queue-size 0 desactiva la cola acotada; los argumentos numéricos no se validan.
- Errores invisibles: las excepciones de los workers se pierden sin traza y provocan pérdida silenciosa de datos.

Esto se debe a que, por un lado, al darle la potestad a la IA de generar el código completo, queriendo cumplir con los requisitos funcionales, en el camino va “asumiendo” cosas para poder llegar al objetivo. En algunas sentencias no se realizan validaciones que pueden ser críticas para el proyecto (por ejemplo, para ingresar la ruta del argumento “--input-dir”, no se valida la existencia del mismo).

También se debe a que, a veces, demasiado detalle en el prompt puede generar que el modelo tome prioridad de algunos requisitos y no toma otros, dejando al modelo de IA utilizado que tome sus propios criterios para crear/modificar/eliminar código.

VI. REFERENCIAS Y BIBLIOGRAFÍA

- [1] M. Agosto 2024. “Python Code Quality Using Radon”. [En línea]. Consultado: 20 de mayo de 2026. Disponible en: <https://medium.com/@chris.alba.hamilton/python-code-quality-using-radon-6f2722ea1eb4>.
- [2] R. Documentation. Mayo 2026. “The cc command”. [En línea]. Consultado: 20 de mayo de 2026. Disponible en: <https://radon.readthedocs.io/en/latest/commandline.html#the-cc-command>.
- [3] M. Learn. Mayo 2026. “Code metrics - Cyclomatic complexity”. [En línea]. Consultado: 20 de mayo de 2026. Disponible en: <https://learn.microsoft.com/es-es/visualstudio/code-quality/code-metrics-cyclomatic-complexity?view=visualstudio>.
- [4] M. Learn. Mayo 2026. “Code metrics - Maintainability index range and meaning”. [En línea]. Consultado: 20 de mayo de 2026. Disponible en: <https://learn.microsoft.com/es-es/visualstudio/code-quality/code-metrics-maintainability-index-range-and-meaning?view=visualstudio>.
- [5] R. Documentation. Mayo 2026. “Introduction to Code Metrics”. [En línea]. Consultado: 20 de mayo de 2026. Disponible en: <https://radon.readthedocs.io/en/latest/intro.html>.
- [6] B. Mayo 2026. “Peak Memory Usage of a Process in Linux”. [En línea]. Consultado: 20 de mayo de 2026. Disponible en: <https://www.baeldung.com/linux/process-peak-memory-usage>.
- [7] M. Learn. Mayo 2026. “Understanding tokens”. [En línea]. Consultado: 20 de mayo de 2026. Disponible en: <https://learn.microsoft.com/es-es/dotnet/ai/conceptual/understanding-tokens>.
- [8] IBM. Mayo 2026. “¿Qué es fine-tuning?”. [En línea]. Consultado: 20 de mayo de 2026. Disponible en: <https://www.ibm.com/mx-es/think/topics/fine-tuning>.
- [9] G. Hub. Mayo 2026. [En línea]. Consultado: 20 de mayo de 2026. Disponible en: <https://github.com/UNLAM-PROG-C/2026-PROGC-Q1-M6>.